



Name of the module: Radix 2 DIT FFT
Location in GIT repository: vspa_common\vspa-lib\fft

Type of code:

- VCPU kernel functions (8 functions)

VCPU Function API:

```
extern void fftDIT<N>_hfl (  
    vspa_complex_float16 const* pIn,           // Input data address.  
    vspa_complex_fixed16* pOut                // Output data address.  
);
```

```
extern void ifftDIT<N>_hfl (  
    vspa_complex_float16 const* pIn,           // Input data address.  
    vspa_complex_fixed16* pOut                // Output data address.  
);
```

```
extern void fftDIT<N>_sfl (  
    vspa_complex_float16 const* pIn,           // Input data address.  
    vspa_complex_float32* pOut                // Output data address.  
);
```

```
extern void ifftDIT<N>_sfl (  
    vspa_complex_float16 const* pIn,           // Input data address.  
    vspa_complex_fixed16* pOut                // Output data address.  
);
```

N = 128, 512

Implementation plan:

This module provides VCPU functions to perform decimation in time (DIT) FFT and IFFT of an input sequence of length as specified by the corresponding N.

The 4 functions are described below.

1. fftDIT<N>_hfl

Accept inputs as half precision floating point complex numbers and writes outputs as half precision fixed point complex numbers.

Mathematically, this function is equivalent to the following MATLAB command:

$$y = (1/N) * \text{fft}(x, N);$$

This is mathematically equivalent to, $y(k) = (1/N) \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$

2. ifftDIT<N>_hfl



Accept inputs as half precision floating point complex numbers and writes outputs as half precision fixed point complex numbers.

This function is equivalent to the following MATLAB command:

$y = \text{ifft}(x, N);$

This is mathematically equivalent to, $y(n) = (1/N) \sum_{k=0}^{N-1} x(k) e^{j2\pi kn/N}$

3. `fftDIT<N>_sfl`

Accept inputs as half precision floating point complex numbers and writes outputs as single precision floating point complex numbers.

Mathematically, this function is equivalent to the following MATLAB command:

$y = \text{fft}(x, N);$

This is mathematically equivalent to, $y(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$

4. `ifftDIT<N>_sfl`

Accept inputs as half precision floating point complex numbers and writes outputs as half precision fixed point complex numbers.

Mathematically, this function is equivalent to the following MATLAB command:

$y = \text{ifft}(x, N);$

This is mathematically equivalent to, $y(n) = (1/N) \sum_{k=0}^{N-1} x(k) e^{j2\pi kn/N}$

In addition to the pointers specified in the function I/O, this function uses an implicit self-allocated scratch buffer, `ifftDITScratchBuffer`, of size $2*N$ words.

The input buffer has to be linear. There is no alignment restriction on the starting address of the input sequence. The output buffer has to be linear and the address must be aligned to the beginning of a DMEM line. The input sequence must be placed in memory in bit-reversed order. The output is in linear order.

The function supports in-place operation but in case of in-place operation the input buffer must be DMEM line aligned as well.

DMEM requirements:

Function name	Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
<code>fftDIT<N>_hfl</code>	Input buffer	N	1	Caller
	Output buffer	N	32	Caller
<code>fftDIT<N>_sfl</code>	Input buffer	N	1	Caller
	Output buffer	$2*N$	32	Caller
<code>ifftDIT<N>_hfl</code>	Input buffer	N	1	Caller
	Output buffer	N	32	Caller



ifftDIT<N>_sfl	Input buffer	N	1	Caller
	Output buffer	N	32	Caller
	Scratch buffer	2*N	32	Caller

N is the FFT/IFFT size.

Target efficiency:

TBD

Test plan:

- **Feature that are going to be tested:**
 - Random Gaussian inputs